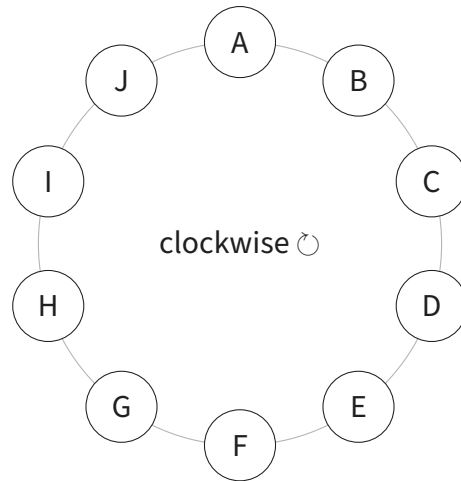


One clue controls the whole code

Name: _____

Date: _____

Our small alphabet is A, B, C, D, E, F, G, H, I, J. To encode, move the **same number of places clockwise** for every letter. After J comes A.



1. Recover the key.

You learn that message A becomes code D. Decode **GDG**. Then encode **BAG**. Check by reversing your moves.

My shift, decoded message, and new code:

2. A clue that cannot fit.

Someone also claims B becomes F under that same key. Could both clues be true? Show why on the ring.

My reason:

Try it with objects. Explain by pointing, talking, or drawing.

A code that skips places

3. Repeat a two-step code.

Start at A. Take two clockwise steps each turn. Write each **landing** until you first return to A. Do not include skipped-over letters. Which letters never appear?

My two-step route and the missing letters:

4. Repeat a three-step code.

Start at A again. Take three steps each turn. Does this rule visit every letter before returning? Give the whole route so someone else can check.

My three-step route:

5. Explain the split.

Start the two-step rule at B instead. How do its landings relate to your route from A? Why can the A-route never jump into the B-route?

Two loops, and why they cannot meet:

Go further: Which step sizes from 1 to 9 visit all ten letters? Can a two-step code still be *decoded* even though one repeated route skips letters?

Try it with objects. Explain by pointing, talking, or drawing.